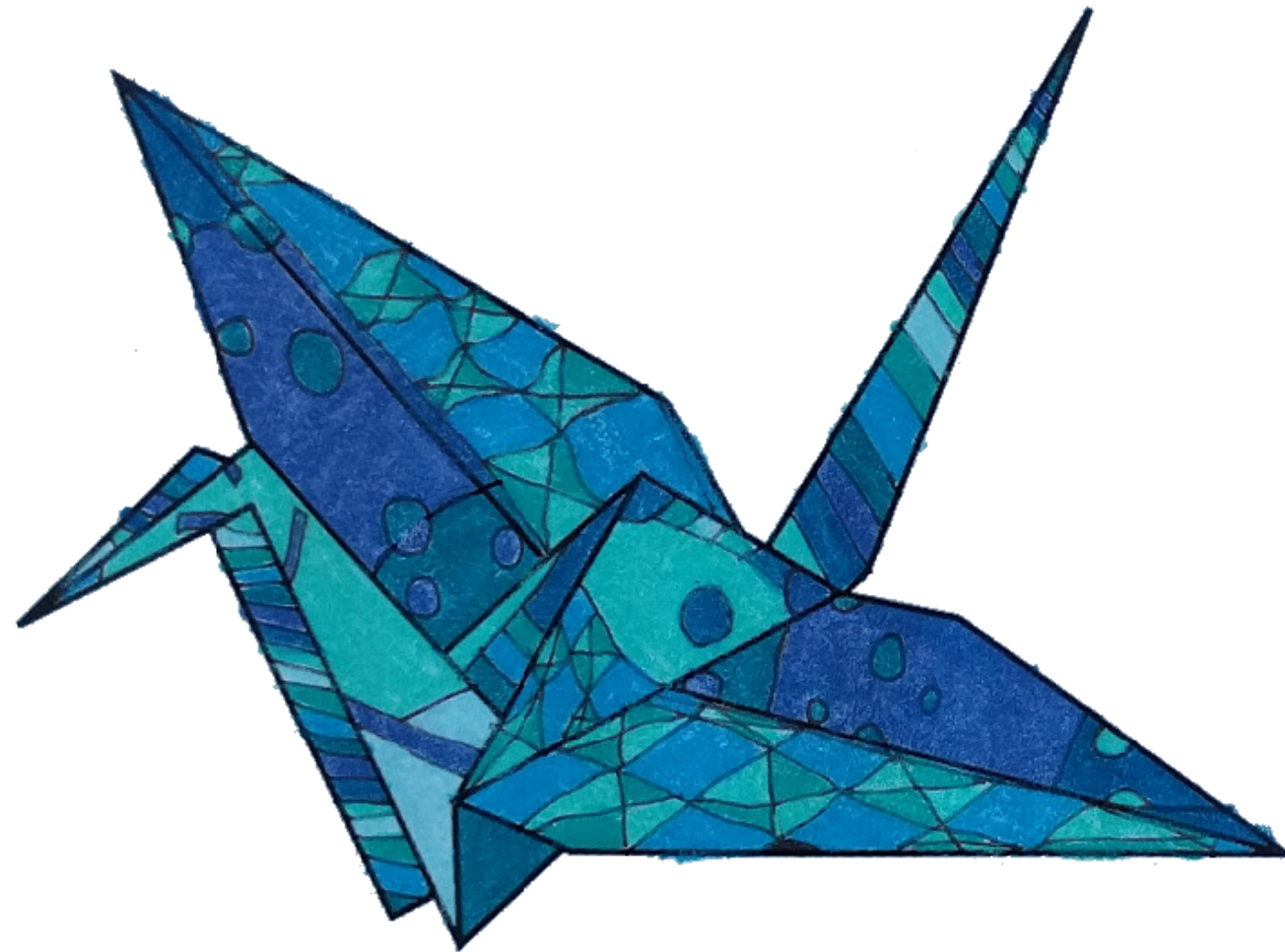


operator<>

A Dream of Higher-Order Metaprogramming Parity



Ben Deane / CppCon Lightning Talks / 2026-09-16

Definition: Functions and Metafunctions

A *function* maps one or more argument values to a return value.

```
auto f(int x, int y) -> int;
```

A *metafunction* does the same thing, but in type-world rather than value-world.

```
template <typename T, typename U>  
using F = std::pair<T, U>;
```

Definition: Higher-Order Function

"A higher-order function is a function that does at least one of the following:

- takes one or more functions as arguments,*
- returns a function as its result."*

-- Wikipedia, Higher Order Functions

Therefore: a higher-order metafunction
either takes metafunction(s) or returns a metafunction.

20 odd years ago...

The C++ community was grappling with higher-order functions (of values).

Because functions aren't really first-class values in C++.

- can't have local definitions
- can't pass/copy/return values
- can't partially apply or easily compose

Our solution at the time: libraries

"If functions can't be first-class values,
let's make first-class values that behave like functions."

```
struct F {  
    int operator()(int x, int y);  
};
```

We know how to define local classes and return the resulting objects.

Library proliferation: Boost.Bind, Boost.Lambda, in-house ad-hoc APIs...

C++11: Enter Lambda Expressions

We saw that the libraries weren't cutting it.

So we put lambda expressions into the language with C++11.

And just like that, higher-order functions got much simpler.

Today: Metaprogramming

We have metaprogramming libraries.

They are amazing. They let us wrangle metafunctions.

But in many ways they are stuck where HOFs were 20 years ago.

Today: Metaprogramming limitations

Templates are not types!

We can't pass types/templates homogeneously.

```
template <????> // parameter: type XOR template (and we must know which)  
using F = ...;  
  
using A = F<int>; // no way  
using B = F<std::optional>; // to do this
```

Today: Metaprogramming limitations

We can't return templates.

```
template <template <typename...> typename G>  
using F = G;           // no way to do this either
```

We can't partially apply templates. We can't compose templates well.

(We *can* actually define templates inside templates.

But that's of limited use because we can't return them!)

Are We in 2006-era HO Metaprogramming?

Perhaps the solution is basically the same as it was for HOFs:

"If metafunctions can't be first-class metavalues,
let's make first-class metavalues that behave like metafunctions."

```
struct pair_with_int {  
    template <typename T>  
    using operator<> = std::pair<int, T>;  
};  
using P = pair_with_int<A>;
```

`operator<>` makes a type behave like a template in the same way that
`operator()` makes an object behave like a function.

Of course you know it Doesn't End There

Now that classes have `operator<>`, we can also have *metalambdas*.

```
using L = [T = int] <typename U> std::pair<T, U>;
```

Like value-lambdas, they have three parts:

- introducer/capture clause: `[T = int]`
- parameter list: `<typename U>`
- body: `std::pair<T, U>`

And you know what the compiler does...

The compiler sees:

```
using L = [T = int] <typename U> std::pair<T, U>;
```

And it synthesizes a class type with `operator<>`:

```
struct __metambda_42 {  
    using T = int;  
    template <typename U>  
    using operator<> = std::pair<T, U>;  
};  
using L = __metambda_42;
```

Metaprogramming Enters the C++11 Era

This is a feature that works the way we already know that value-lambdas do.

Now we can:

- pass metavalues and metafunctions homogeneously
- actually return metafunctions!
- compose and partially apply metafunctions much more simply

But There's an Elephant in the Room

Hold on Ben, what about Reflection?
(For that matter, what happened to Universal Template Parameters?)

Sure, you can achieve some of these effects with reflection, but:

- it's limited to value-world
- your code is littered with `^^` (and `dealias`, right?)
- we don't have `constexpr` function params, so you *have* to stay in value-world

And Most Importantly

Reflection can only reflect (ha) what the language allows.

In this sense, reflection *complements* template metaprogramming.

It doesn't *compete with* it or *replace* it.

It can make it *easier*, but not *simpler*.

There's more...

At this point, some of you want this feature.

And some of you hate it.

(Do you hate the *idea* or just the *syntax*?)

Well, you thought *that* was a controversial idea?

We still have 3 slides left...

Overload sets

Once we have easy metafunction wrangling,
we want to do more things that value-functions can do.

Like overloading.

```
template <typename T, typename U>           // basically  
using F = std::pair<T, U>;                 // no type-based form  
                                           // of this  
  
template <typename... Ts>                   // works today  
    requires (sizeof...(Ts) != 2)           // but it would be so useful  
using F = std::tuple<Ts...>;               // don't you think?
```

And if we have overloads, we need a way to resolve them...

What about ADL?

The final boss of metafunction parity?

Maybe you like ADL, maybe you hate it.
It exists, and it does useful things sometimes.

This Could Happen! C++...32? 35?

(This probably won't happen.)

But it is:

- a relatively simple idea
- that works analogously to what we already know
- and smooths a *bunch* of infelicities

So we can dream...